

NEXORA

AI-Powered Student Productivity & Career Intelligence Platform

Final Architecture & Implementation Notes — v4 (2-Month Build Plan)

FINAL DECISIONS: JavaScript-only backend; Node.js + Express; MongoDB + Mongoose; hosted pre-trained LLM API behind a provider-agnostic adapter; semantic/vector memory is Phase 1.5, not a Week-1 dependency.

This final revision keeps the JavaScript-only Node.js + Express + MongoDB architecture and adds the final hardening decisions: explicit refresh-token storage/revocation, concrete AI budget and rate limits, privacy-safe fresh chats, a bounded context window, prompt-injection hygiene, AI usage telemetry, account deletion, a promoted end-to-end test, and a clear schedule cut-line that protects authentication, tests, security, and deployment.

Table of Contents

- 1. Final Technology Decision
- 2. Core System Mental Model
- 3. Full Recommended Project Structure
- 4. Responsibility of Each Layer
- 5. REST API Map
- 6. MongoDB Data Model
- 7. AI + Memory Pipeline
- 8. Memory Controls & Privacy
- 9. Feedback-Aware Recommendations
- 10. Testing Strategy
- 11. Performance & Scaling
- 12. Security Must-Haves
- 13. CI/CD & Deployment
- 14. Two-Month Development Roadmap
- 15. What to Build vs What Not to Build
- 16. Phase 2 / Stretch Goals
- 17. Resume & Viva Description
- 18. Final Architecture Summary

1. Final Technology Decision

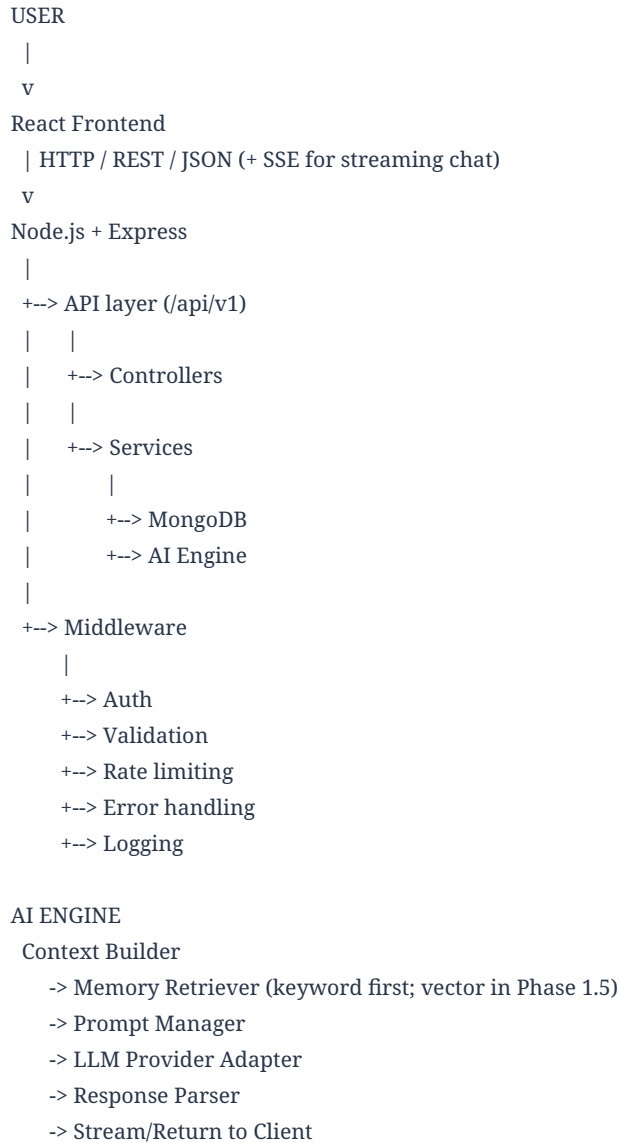
Layer	Technology	Purpose	Decision
Frontend	React + Vite	UI, dashboards, chat, task/study/career screens	FINAL
Styling	Tailwind CSS	Fast, consistent, responsive UI	FINAL
Charts	Recharts	Productivity, study, skill analytics	FINAL
Backend	Node.js + Express.js	Server, business logic, REST APIs	FINAL
Language	JavaScript	Single backend language; easier scope for the team	FINAL
Database	MongoDB + Mongoose	Users, tasks, study, projects, chats, memories	FINAL
Authentication	JWT access + refresh + Argon2id/bcrypt	Login and protected APIs	FINAL
API	REST / JSON, versioned /api/v1	Frontend-backend communication	FINAL
AI	Pre-trained LLM API	Chat, recommendations, roadmaps, insights	FINAL
AI integration	Provider-agnostic adapter	Swap LLM provider without rewriting app logic	FINAL
Semantic memory	MongoDB Atlas Vector Search or lightweight vector store	Relevant long-term memory by meaning	PHASE 1.5
Streaming	SSE	Token-by-token chat response	POLISH
Testing	Jest + Supertest + React Testing Library	Backend and frontend tests	FINAL
CI/CD	GitHub Actions	Lint/test checks on pull requests	FINAL
Docs	OpenAPI/Swagger + Postman	API documentation and testing	FINAL
Deployment	Vercel + Render/Railway + MongoDB Atlas	Simple production deployment	FINAL
Monitoring	Sentry + structured logging	Find production errors quickly	POLISH
AI cost control	Per-user budget + token cap + circuit breaker	Prevent runaway LLM usage and demo-day cost spikes	FINAL
Refresh-token storage	httpOnly Secure cookie + hashed server-side auth session	Rotation, revocation, logout, and reuse detection	FINAL

LLM DECISION: Use a hosted pre-trained LLM API for the main build. Do not self-host model weights in NEXORA for this timeline. The resume value comes from context building, memory, recommendation logic, API design, security, testing, and deployment—not from managing GPU infrastructure.

Why this stack?

- React gives you a strong, modern student-facing UI and dashboard experience.
- Node.js + Express keeps the entire application in JavaScript, matching your current learning direction.
- MongoDB is a natural fit for flexible user profiles, conversations, messages, memories, and recommendation documents.
- A hosted LLM keeps the AI feature practical while the adapter design preserves the option to switch providers later.
- The architecture remains modular without becoming a microservices project.

2. Core System Mental Model



The key rule is: React never talks directly to MongoDB or the LLM provider. Every private operation goes through the Node.js backend, where authentication, validation, business rules, memory retrieval, and AI orchestration are controlled.

Typical non-AI request



Typical AI request

React

- > POST /api/v1/ai/chat
- > ai.controller.js
- > ai.service / conversation service
- > memoryRetriever.js
- > MongoDB (profile + recent messages + relevant memories)
- > contextBuilder.js
- > promptManager.js
- > llmClient.js
- > hosted LLM API
- > responseParser.js
- > SSE / JSON response
- > React
- > save user + assistant messages
- > optionally save important memory

3. Full Recommended Project Structure

```

NEXORA/
|
├── frontend/                # React + Vite
|   ├── public/
|   ├── src/
|   |   ├── assets/
|   |   ├── components/
|   |   |   ├── common/
|   |   |   |   ├── Button.jsx
|   |   |   |   ├── Input.jsx
|   |   |   |   ├── Modal.jsx
|   |   |   |   ├── Loader.jsx
|   |   |   |   └── ErrorMessage.jsx
|   |   |   └── layout/
|   |   |       ├── Navbar.jsx
|   |   |       ├── Sidebar.jsx
|   |   |       └── PageLayout.jsx
|   |   ├── dashboard/
|   |   |   ├── ProductivityCard.jsx
|   |   |   ├── TaskOverview.jsx
|   |   |   ├── SkillProgress.jsx
|   |   |   ├── StudyOverview.jsx
|   |   |   └── AIInsight.jsx
|   |   ├── tasks/
|   |   ├── study/
|   |   ├── career/
|   |   ├── projects/
|   |   └── chat/
|   |       ├── ChatWindow.jsx
|   |       ├── MessageBubble.jsx
|   |       ├── ChatInput.jsx
|   |       ├── ChatHistory.jsx
|   |       └── MemoryIndicator.jsx
|   |   └── pages/
|   |       ├── Login.jsx
|   |       ├── Register.jsx
|   |       ├── Dashboard.jsx
|   |       ├── Tasks.jsx
|   |       ├── Study.jsx
|   |       ├── Career.jsx
|   |       ├── Projects.jsx
|   |       ├── Analytics.jsx
|   |       ├── AIChat.jsx
|   |       └── Settings.jsx
|   |   └── services/
|   |       ├── api.js
|   |       ├── auth.service.js
|   |       ├── task.service.js
|   |       └── study.service.js

```

```

| | | └─ career.service.js
| | | └─ project.service.js
| | | └─ ai.service.js
| | └─ context/
| |   └─ AuthContext.jsx
| |   └─ ChatContext.jsx
| | └─ hooks/
| | └─ utils/
| | └─ routes/
| | └─ App.jsx
| | └─ main.jsx
| └─ tests/
| └─ package.json
| └─ vite.config.js
|
└─ backend/                                # Node.js + Express + JavaScript
  └─ src/
    └─ server.js
    └─ app.js
    └─ api/
      └─ v1/
        └─ auth/
          └─ auth.routes.js
          └─ auth.controller.js
          └─ auth.validator.js
        └─ users/
          └─ user.routes.js
          └─ user.controller.js
          └─ user.validator.js
        └─ tasks/
        └─ study/
        └─ career/
        └─ projects/
        └─ analytics/
        └─ conversations/
        └─ memory/
        └─ ai/
        └─ health/
      └─ services/
        └─ auth.service.js
        └─ user.service.js
        └─ task.service.js
        └─ study.service.js
        └─ career.service.js
        └─ project.service.js
        └─ analytics.service.js
        └─ conversation.service.js
        └─ memory.service.js
        └─ recommendation.service.js
      └─ models/
        └─ User.model.js

```

```

| | | └─ Task.model.js
| | | └─ StudySession.model.js
| | | └─ Skill.model.js
| | | └─ CareerGoal.model.js
| | | └─ Project.model.js
| | | └─ Conversation.model.js
| | | └─ Message.model.js
| | | └─ Memory.model.js
| | | └─ Recommendation.model.js
| | └─ ai/
| | | └─ providers/
| | | | └─ openai.provider.js
| | | | └─ anthropic.provider.js # optional second provider
| | | └─ llmClient.js
| | | └─ contextBuilder.js
| | | └─ promptManager.js
| | | └─ memoryRetriever.js
| | | └─ embeddings.js # Phase 1.5
| | | └─ responseParser.js
| | └─ middleware/
| | | └─ auth.middleware.js
| | | └─ error.middleware.js
| | | └─ notFound.middleware.js
| | | └─ rateLimit.middleware.js
| | | └─ validation.middleware.js
| | | └─ requestLogger.middleware.js
| | └─ config/
| | | └─ db.js
| | | └─ env.js
| | | └─ ai.js
| | └─ utils/
| | | └─ logger.js
| | | └─ jwt.js
| | | └─ hashing.js
| | | └─ helpers.js
| | └─ seed/
| | | └─ careers.seed.js
| | | └─ skills.seed.js
| | | └─ projects.seed.js
| └─ tests/
| └─ package.json
| └─ .env
|
└─ data/
  └─ careers.json
  └─ skills.json
  └─ roadmaps.json
  └─ project_templates.json
|
└─ docs/
  └─ architecture.md

```

```
| ├── api.md
| ├── database.md
| └── adr/
|
| ├── .github/
| │   ├── workflows/
| │   └── ci.yml
|
| ├── .gitignore
| ├── README.md
| └── docker-compose.yml
```

IMPORTANT: The old local `ai_models/pretrained_model` directory is intentionally removed. NEXORA uses an LLM API through `ai/llmClient.js`. Semantic embeddings/vector search are a Phase 1.5 enhancement, not a Week-1 dependency.

4. Responsibility of Each Layer

Layer	Responsibility	Do NOT put here
React components/pages	UI, interaction, rendering	MongoDB access, secrets, core business rules
API routes	URL + HTTP method mapping	Large business logic
Controllers	Read request, validate, call service, return response	Database-heavy logic
Services	Business rules and orchestration	UI code
Models	MongoDB schema/persistence mapping	AI prompt logic
Middleware	Auth, validation, rate limiting, errors, logging	Feature-specific business logic
AI folder	Provider adapter, context, prompts, memory retrieval, embeddings	UI or random CRUD
Config	Environment, DB, AI provider settings	Hard-coded secrets

Example: Create a task

React Tasks page
 -> task.service.js
 -> POST /api/v1/tasks
 -> task.routes.js
 -> task.controller.js
 -> task.service.js
 -> Task.model.js
 -> MongoDB
 -> JSON response
 -> React

Example: AI chat

React AIChat.jsx
 -> ai.service.js
 -> POST /api/v1/ai/chat
 -> ai.controller.js
 -> conversation.service.js
 -> memory.service.js / memoryRetriever.js
 -> contextBuilder.js
 -> promptManager.js
 -> llmClient.js
 -> LLM provider
 -> responseParser.js
 -> SSE / JSON
 -> React
 -> save conversation + optional memory

5. Recommended REST API Map

Module	Method	Endpoint	Purpose
AUTH	POST	/api/v1/auth/register	Create account
AUTH	POST	/api/v1/auth/login	Login and issue access + refresh token
AUTH	POST	/api/v1/auth/refresh	Rotate access token
AUTH	POST	/api/v1/auth/logout	Invalidate refresh session
AUTH	GET	/api/v1/auth/me	Current user
USERS	GET/PUT	/api/v1/users/me	Read/update profile
TASKS	GET/POST	/api/v1/tasks	List/create tasks
TASKS	PUT/DELETE	/api/v1/tasks/:id	Update/delete task
STUDY	GET/POST	/api/v1/study/sessions	List/log study session
CAREER	GET	/api/v1/career	Career profile
CAREER	POST	/api/v1/career/skill-gap	Skill-gap analysis
CAREER	POST	/api/v1/career/roadmap	Generate roadmap
PROJECTS	GET/POST	/api/v1/projects	List/create projects
ANALYTICS	GET	/api/v1/analytics/dashboard	Dashboard metrics
ANALYTICS	GET	/api/v1/analytics/productivity	Productivity metrics
CHAT	GET/POST	/api/v1/conversations	List/create chats
CHAT	GET/POST	/api/v1/conversations/:id/messages	Load/send messages
CHAT	GET	/api/v1/conversations/:id/stream	SSE stream of AI response
MEMORY	GET/POST	/api/v1/memory	View/create memory
MEMORY	DELETE	/api/v1/memory/:id	Delete one memory
MEMORY	DELETE	/api/v1/memory/all	Delete all memories
AI	POST	/api/v1/ai/chat	AI conversation
AI	POST	/api/v1/ai/recommend	Personalized recommendation
AI	POST	/api/v1/ai/generate-roadmap	AI roadmap
AI	POST	/api/v1/ai/skill-analysis	AI skill analysis
SYSTEM	GET	/api/v1/health	Liveness/readiness check
USERS	DELETE	/api/v1/users/me	Delete account and user-owned data
AI USAGE	GET	/api/v1/ai/usage	View current-user AI usage/cost metrics

API RULE: List endpoints should support pagination such as ?page=1&limit=30. Scope every protected query to the authenticated user. Keep /api/v1 as the stable external contract.

6. MongoDB Data Model

Collection	Key data	Why it exists
users	name, email, passwordHash, preferences, careerGoal	Identity + profile
tasks	userId, title, status, priority, deadline, timestamps	Productivity tracking
studySessions	userId, subject, duration, date, notes	Study history
skills	name, category, level	Reference skills
careerGoals	userId, targetRole, targetDate, requiredSkills	Career planning
projects	userId, title, techStack, status	Project tracking
conversations	userId, title, timestamps	Chat sessions
messages	conversationId, role, content, timestamp	Conversation history
memories	userId, memory, type, importance, embedding?, timestamps	Long-term memory; embedding optional in Phase 1.5
recommendations	userId, recommendation, category, feedback, status	Feedback-aware recommendations
authSessions	userId, tokenHash, familyId, expiresAt, revokedAt, createdAt	Hashed refresh-token storage, rotation, revocation, and reuse detection
aiUsage	userId, provider, model, latencyMs, inputTokens, outputTokens, estimatedCost, success, createdAt	Usage, latency, and cost telemetry for optimization and viva metrics

Recommended indexes

- { userId: 1, createdAt: -1 } for tasks, studySessions, memories, recommendations where relevant.
- { conversationId: 1, timestamp: -1 } for recent chat retrieval.
- { userId: 1, title: 1 } or other targeted indexes only when supported by real queries.
- Vector index on memories.embedding only when MongoDB Atlas Vector Search is enabled in Phase 1.5.

Data ownership rule

A user must never be able to query another user's private tasks, study sessions, conversations, memories, or recommendations. Every service method should receive the authenticated userId and scope database queries accordingly.

7. AI + Memory Pipeline

1. User sends a message from React or starts an AI action.
2. `ai.routes.js` receives the request.
3. `ai.controller.js` validates input and calls the relevant service.
4. `memoryRetriever.js` finds relevant long-term memories. MVP: keyword/category filters. Phase 1.5: vector similarity re-ranking.
5. `conversation.service.js` retrieves only recent, bounded conversation history—not the full chat database.
6. `contextBuilder.js` combines the profile, skills, activity, relevant memories, recent messages, and current request.
7. `promptManager.js` creates the final model input, including memory-use rules and a predictable response format.
8. `llmClient.js` calls the configured hosted LLM provider through the adapter.
9. `responseParser.js` validates/normalizes the model output and supports streaming where implemented.
10. `conversation.service.js` saves the user and assistant messages.
11. `memory.service.js` optionally stores important user facts with an importance/relevance score.
12. The backend returns the final response or closes the SSE stream.

The critical concept

NEXORA DOES NOT “MAGICALLY REMEMBER”: MongoDB stores the user’s durable data; the memory retriever selects relevant facts; the context builder places those facts into the LLM request. The LLM provides generation, while NEXORA provides continuity.

Three kinds of memory

Type	Meaning	Example
Short-term	Current conversation context	“What does this mean?” after the previous message
Session	Facts relevant to the current chat	Current topic: React hooks
Long-term	Important user facts saved across chats and retrieved by relevance. Fresh/private chat excludes long-term reads and writes.	Name, career goal, preferences, rejected ideas

8. Memory Controls & Privacy

Make memory a first-class product feature rather than an invisible backend trick. The Settings page should expose understandable controls so users can decide whether NEXORA uses saved context.

- AI Memory: ON/OFF
- Remember profile: ON/OFF
- Remember conversations: ON/OFF
- Remember preferences: ON/OFF
- View saved memories
- Delete one memory
- Delete all memories
- Start a fresh chat that ignores long-term memory
- Export memories as JSON (optional polish feature)

Fresh-idea behavior

User: "Give me a project idea."

AI: "Build an AI Resume Analyzer."

User: "No. Give me something completely different."

Backend adds context:

- previous recommendation = rejected
- instruction = avoid previously rejected/seen ideas
- instruction = produce a meaningfully different category

LLM -> new recommendation

Privacy rule

FRESH CHAT / PRIVATE CHAT: When the user chooses not to use memory, contextBuilder.js must omit long-term memories and other disallowed context. The session must also set saveMemory=false so memory.service.js does not create or update long-term memories from that chat. For privacy, the model receives only the information necessary for the current request.

9. Feedback-Aware Recommendations

AI Recommendation

```

|
+--> Helpful
|   -> store positive feedback
|
+--> Not useful
|   -> store negative feedback
|
+--> Give another
|   -> retrieve rejected/seen items
|   -> build “do not repeat” context
|   -> generate a different recommendation

```

Recommendation data

Field	Example
userId	user_123
recommendation	Build an AI Resume Analyzer
category	career-project
feedback	negative
status	rejected
createdAt	timestamp

Later, track acceptance rate by category. This gives NEXORA a measurable product metric such as “career-project recommendations accepted by users” instead of only making qualitative claims about AI quality.

- Store recommendation feedback as explicit data (helpful, not useful, rejected, accepted). Do not infer preference only from chat text.

10. Testing Strategy

Area	Tooling	What to cover
Backend unit/integration	Jest + Supertest	Auth, tasks, career, memory, AI routes, recommendation logic
Frontend	React Testing Library + Jest	Forms, dashboard components, chat rendering, protected-route redirects
AI pipeline	Mocked provider	Prompt building, memory filters, response parsing, adapter contract
E2E (should build)	Playwright	Login -> create task -> chat -> verify AI response -> verify memory/privacy behavior -> verify dashboard effect

CI RULE: Never call the real LLM provider from CI tests. Mock the provider adapter so tests are deterministic, fast, and free of provider/API-cost dependencies.

Minimum test targets

- Registration/login and protected routes
- Task CRUD and user ownership
- Career skill-gap endpoint
- Conversation + memory retrieval
- AI adapter and response parsing
- Recommendation rejection / “give another” flow
- One Playwright end-to-end smoke flow: login -> create task -> chat -> verify AI response -> verify memory/privacy behavior -> verify dashboard effect.
- CI must use a deterministic stubbed/mock LLM provider; never call the paid/remote LLM provider from CI tests.

11. Performance & Scaling

- Reuse a single LLM client/configuration rather than creating a new client object for every request.
- Never send the entire database or the entire chat history to the model; retrieve only relevant, bounded context.
- Add MongoDB indexes for userId, conversationId, createdAt, and other fields based on real query patterns.
- Paginate chat history, tasks, study sessions, and memory lists.
- Keep normal analytics calculations in JavaScript/backend logic instead of asking the LLM to calculate basic numbers.
- Cache repeated reference reads only if a bottleneck appears; Redis is optional, not a core dependency.
- Apply two rate-limit tiers: a general API limiter for normal CRUD traffic and a stricter AI limiter tied to cost controls. Initial AI defaults: 20 AI requests per user per day and a maximum of 1000 generated tokens per response; tune only after measuring real provider cost.
- Add a hard circuit breaker when the per-user daily budget, provider quota, or repeated provider-failure threshold is reached. Return a clear retry/budget message instead of repeatedly calling the provider.
- Log every AI request with provider, model, latencyMs, inputTokens, outputTokens, estimatedCost, success/failure, userId, and timestamp. Use this data for latency/cost monitoring and viva metrics.
- Keep V1 AI context bounded to: current request + relevant profile/context + the last 15 messages of the active conversation. Older messages are not sent to the model; summarization is future work.
- Use SSE streaming for AI chat after the non-streaming version is stable so perceived latency is low.

What makes NEXORA feel fast?

User sends message

- > retrieve small context
- > send request to LLM
- > optionally stream tokens with SSE after stable JSON chat works
- > render incrementally

Instead of:

User sends message

- > load everything
- > wait for complete response
- > render once

12. Security Must-Haves

- Keep API keys, MongoDB URLs, JWT secrets, and provider settings in .env. Never commit them.
- Never expose the LLM/provider secret in React. All AI calls go through the backend.
- Hash passwords with Argon2id (preferred) or bcrypt.
- Use short-lived JWT access tokens plus refresh-token rotation. Store the refresh token in an httpOnly + Secure + appropriate SameSite cookie; store only a hash of the refresh token server-side in authSessions; rotate on every use; revoke on logout; detect refresh-token family reuse and revoke that family.
- Protect private routes with authentication middleware.
- Always scope MongoDB queries to the authenticated user.
- Validate and sanitize incoming data before business logic runs.
- Use separate rate limits for general APIs and AI APIs. Example starting defaults: 100 general requests per 15 minutes per IP/user plus the AI budget of 20 requests/day/user; tune after real traffic testing.
- Prompt-injection hygiene: clearly delimit system instructions, user input, retrieved memory, and conversation text. Never treat stored memory as an instruction, and never place provider secrets or private credentials into prompts.
- Validate and sanitize AI output before rendering. If Markdown/HTML is supported, use a safe renderer/sanitizer and never inject untrusted raw HTML into the DOM.
- Add global error handling, structured logging, and rate limiting.
- Keep .env, node_modules, credential files, and secrets out of Git.

Do not do this

React -> MongoDB directly X

React -> LLM provider directly X

Hard-coded API key in frontend X

Plain-text passwords X

One huge controller with all business logic X

Store every chat sentence as permanent memory by default X

- Every protected query must be scoped to the authenticated userId. Cross-user reads must be impossible by default at the service layer, not only in the UI.
- Account deletion: DELETE /api/v1/users/me must remove or safely anonymize user-owned tasks, study sessions, projects, conversations, messages, memories, recommendations, auth sessions, and AI usage records according to the project data policy.

13. CI/CD & Deployment

Component	Target	Notes
Backend API	Render or Railway	Simple demo deployment with environment variables
Frontend	Vercel	Convenient React deployment and preview builds
Database	MongoDB Atlas	Managed MongoDB; avoid local-only dependency for the public demo
CI	GitHub Actions	Install -> lint -> test -> block merge on failure
Monitoring	Sentry + structured logs	Useful when the deployed demo behaves unexpectedly

Local development	Docker Compose	Optional parity for frontend/backend/database; not a Week-1 blocker
-------------------	----------------	---

CI workflow

Pull Request

- > install dependencies
- > lint
- > backend tests
- > frontend tests
- > pass/fail status

Merge to main

- > deploy backend
- > deploy frontend

- Merge is blocked when lint/tests fail. The real LLM provider is never called from CI tests.
- Deploy only after the end-to-end smoke flow passes against the deployed environment or a production-like staging target.

14. Two-Month Development Roadmap

Week	Focus	Expected result
1	Project setup, Git workflow, React shell, Express skeleton, MongoDB, JWT auth, CI	Register/login/protected route working end-to-end; CI green
2	Tasks + Study + Dashboard shell	Core CRUD works without AI; first analytics widgets live
3	Career + Projects + Analytics	Non-AI product usable and demoable
4	Chat UI + conversation storage + LLM provider adapter	Working AI chat with no memory yet
5	Short/session/long-term memory + memory controls + contextBuilder	Personalized AI chat across sessions
6	Embeddings/vector retrieval + SSE streaming (Phase 1.5; only if the core is stable)	Optional semantic retrieval + live token streaming; cut these first if the schedule slips
7	Feedback-aware recommendations + skill-gap + roadmap generation	AI features connected to career/skills data
8	Polish, accessibility, Playwright smoke test, docs, deployment, demo video	E2E-verified, deployed, resume-ready project with clear demo evidence

BUILD STRATEGY: Work in end-to-end vertical slices. Do not finish the entire frontend before starting the backend, and do not build the AI system before the core product exists. Keep a CHANGELOG.md; it becomes useful evidence of engineering progress for GitHub, viva, and resume writing.

15. What to Build vs What Not to Build

MUST BUILD — core NEXORA

- React dashboard and responsive UI
- Node.js + Express REST API with /api/v1 versioning
- MongoDB + Mongoose data layer
- JWT authentication + secure password hashing
- Tasks, Study, Career, Projects, Analytics
- Conversation history
- Three-level memory: short-term, session, long-term
- Context builder and personalized AI chat
- AI skill-gap analysis and career roadmap
- Feedback-aware “Helpful / Not useful / Give another” recommendations
- Validation, error handling, rate limiting, tests, API documentation
- Deployment and README with architecture diagram
- Account deletion endpoint and user-owned data cleanup
- AI usage logging: latency, token usage, estimated cost, provider/model, success/failure

SHOULD BUILD — polish

- SSE streaming for chat
- Memory settings UI
- Sentry/error tracking
- GitHub Actions CI
- One Playwright end-to-end smoke test
- Accessibility pass
- Empty/loading/error states
- Good responsive design
- Recommendation acceptance-rate metric

DO NOT BUILD IN CORE TIMELINE

- Training an LLM from scratch
- Self-hosting large model weights
- Microservices
- Kubernetes
- Kafka
- Complex Celery/job-queue infrastructure
- Multiple databases
- Huge multi-agent architecture
- Redis before a real bottleneck exists
- Vector database before normal MongoDB memory works
- Do not build vector memory and SSE in parallel with the first working AI chat; they are Phase 1.5/polish only.

SCOPE: If behind schedule, cut SSE/vector search first; never cut auth, security, tests, or deployment.

16. Phase 2 / Stretch Goals

- RAG over the career/skills/project knowledge base for more grounded roadmap generation
- Redis caching for hot reads if profiling shows a bottleneck
- Push notifications or email digests for deadlines and study streaks
- A/B testing of recommendation prompts
- Basic admin dashboard for seed/monitoring
- More provider adapters if you want to demonstrate provider portability
- Advanced semantic memory management

Phase 1.5: Semantic Memory

User message

- > memory candidate extraction
- > embedding generation
- > MongoDB memory document + embedding
- > vector index

Future user question

- > embed query
- > vector similarity search
- > top relevant memories
- > contextBuilder.js
- > LLM

Only move to this layer after the regular conversation + long-term memory flow is stable. The goal is to demonstrate semantic retrieval, not to introduce infrastructure for its own sake. If the schedule slips, skip this phase without changing the core architecture.

17. Resume & Viva Description

RECOMMENDED RESUME DESCRIPTION: NEXORA is a full-stack AI productivity and career intelligence platform built with React, Node.js/Express, MongoDB, and a hosted pre-trained LLM API behind a provider-agnostic adapter. It features JWT authentication with refresh-token rotation, user-controlled short/session/long-term memory, bounded context retrieval, feedback-aware recommendations, skill-gap analysis, AI-generated roadmaps, REST APIs, automated testing, CI/CD, AI usage telemetry, and production deployment.

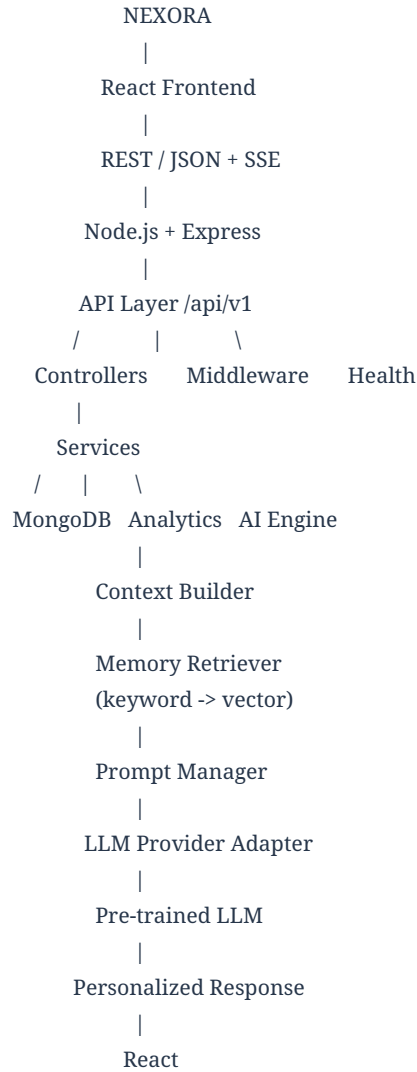
How to explain the architecture in a viva

- “React is responsible for the user experience. It never talks directly to the database or LLM provider.”
- “Node.js + Express exposes versioned REST APIs and contains the application business logic.”
- “Controllers handle HTTP concerns; services handle business rules; Mongoose models handle MongoDB persistence.”
- “NEXORA memory is stored in MongoDB. A memory retriever chooses relevant context instead of sending the entire history to the LLM.”
- “The LLM is a pre-trained hosted model. NEXORA adds value through context building, memory, recommendations, skill-gap logic, and product workflows.”
- “Semantic/vector memory is an enhancement after the basic memory system works.”
- “The provider adapter lets us switch LLM providers without rewriting the rest of the application.”

What separates NEXORA from a generic chatbot?

- Persistent user profile + career goal
- Conversation continuity
- User-controlled memory
- Skill and productivity data used as AI context
- Feedback-aware recommendations
- Career skill-gap analysis
- Project recommendations
- AI-generated personalized roadmaps
- A full REST backend with authentication, persistence, testing, CI/CD, and deployment

18. Final Architecture Summary



Final technology checklist

- Frontend: React + Vite + Tailwind + Recharts
- Backend: Node.js + Express + JavaScript
- Database: MongoDB + Mongoose
- API: REST + /api/v1
- Auth: JWT + Argon2id/bcrypt
- AI: hosted pre-trained LLM API behind an adapter
- Memory: MongoDB first; semantic/vector retrieval in Phase 1.5
- Chat UX: normal JSON first; SSE streaming after stable chat
- Testing: Jest + Supertest + React Testing Library
- Documentation: OpenAPI/Swagger + Postman
- CI/CD: GitHub Actions
- Deployment: Vercel + Render/Railway + MongoDB Atlas
- Optional polish: SSE, Sentry, Redis, Docker Compose, RAG, semantic/vector memory

FINAL RECOMMENDATION: Freeze the architecture. Build the end-to-end core first; add SSE, vector search, Redis, or other advanced infrastructure only after the core works.